Worker threads

10 min read Updated 24 Jul 2026 Topic 62 of 70

Why it matters

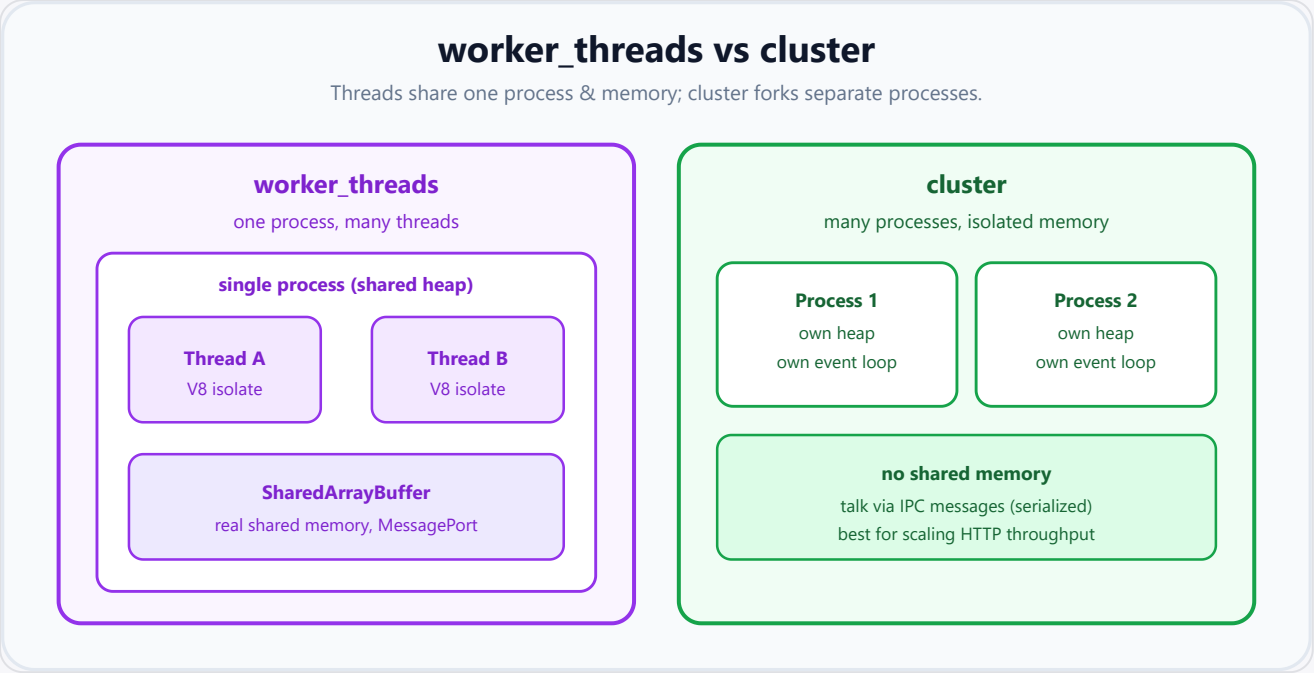
Node's event loop is fast at one thing above all: juggling *waiting*. Reading files, calling databases, awaiting HTTP — none of that occupies the CPU, so a single thread can shepherd thousands of concurrent I/O operations. But the moment a request needs to *compute* — hash a password, resize an image, parse a 20 MB CSV, run a regex over huge input, do cryptography or ML inference — that work runs **on the main thread**, and while it runs the event loop is frozen. Every other in-flight request stalls until the computation finishes. This is the infamous "blocking the event loop," and it is the number-one way a Node service goes from snappy to unresponsive under real workloads.

The **worker_threads** module (stable since Node 12) is the correct tool for this. It lets you run JavaScript on *additional* threads inside the same process, so CPU-heavy work happens off the main thread and the event loop stays free to serve requests. In interviews, "how do you handle CPU-intensive work in Node without blocking?" is a staple, and the crisp answer — worker_threads for CPU-bound work, and here's when it beats cluster — is exactly what distinguishes a senior answer from "just use async/await" (which does nothing for CPU-bound code).

The concept, simply

A worker thread is a **separate JavaScript execution context running on its own OS thread, inside the same process**. Each worker gets its own V8 *isolate*: its own event loop, its own micro/macro task queues, and — crucially — its own heap. So by default, two threads do *not* share variables; each has its own copy of everything. This makes worker_threads safe by default: you don't get the classic multithreaded data races on ordinary objects, because ordinary objects aren't shared.

copied using the structured clone algorithm. If copying a big payload is too expensive, you have two escape hatches: **transfer** an `ArrayBuffer` (hand over ownership with zero copy — the sender loses access), or use a **SharedArrayBuffer**, which is genuinely shared memory both threads can read and write live (coordinated with `Atomics`).



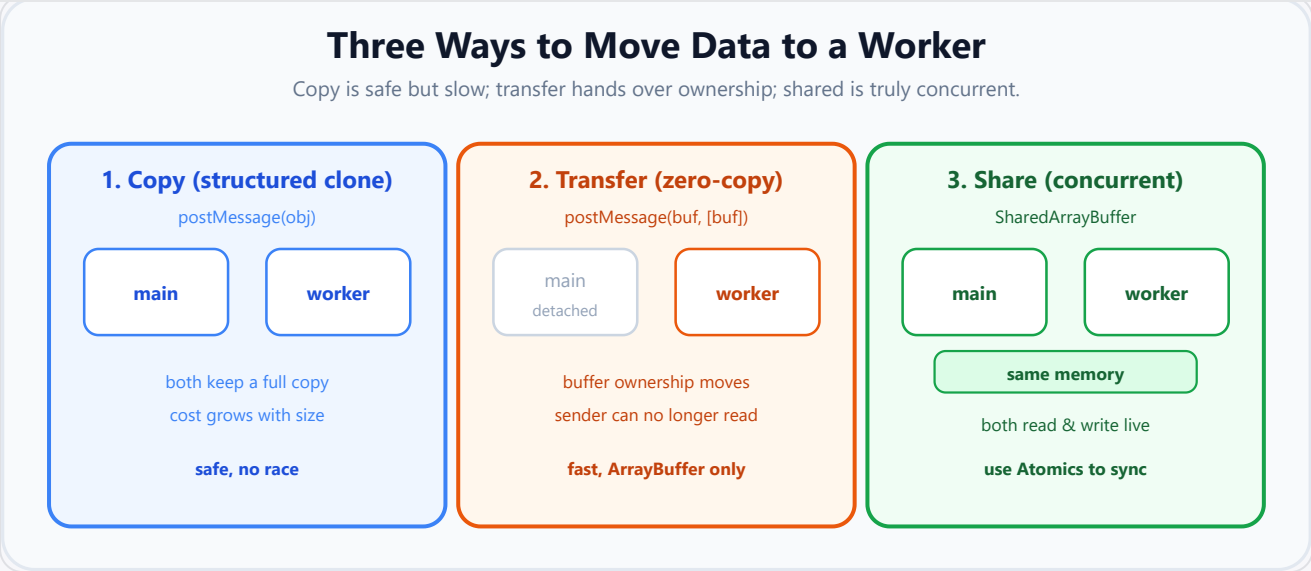
worker_threads live in one process and can share memory; cluster spawns separate processes that share nothing but a port.

The key distinction from cluster: **cluster is many processes for I/O throughput; worker_threads is many threads for CPU work.** Cluster answers "serve more simultaneous connections across cores." Worker_threads answers "do this heavy calculation without freezing the request handler." They are complementary, not competitors — a big service might cluster for concurrency *and* use worker_threads (or a pool of them) for the occasional CPU-bound job.

How it works

You create a worker with `new Worker(filename, { workerData })`. Node spins up a new thread, creates a fresh V8 isolate on it, and runs the given file from scratch — modules are *not* shared, so the worker re-imports what it needs. The `workerData` value is structured-cloned into the worker at start-up as its initial input. From then on, the two sides exchange messages over the port.

Data movement has three modes, and understanding their cost is the whole game:



Copy is safe but scales with payload size; transfer hands over an ArrayBuffer with zero copy; SharedArrayBuffer is real shared memory coordinated with Atomics.

- **Copy (default):** `postMessage(obj)` deep-clones the object. Simple and safe, but the cost grows with size — cloning hundreds of MB defeats the purpose.
- **Transfer (zero-copy):** `postMessage(buf, [buf.buffer])` moves an `ArrayBuffer`'s ownership to the other thread without copying bytes. The sender's buffer becomes *detached* (unusable). Ideal for handing a large binary payload one-way.
- **Share (concurrent):** a `SharedArrayBuffer` is mapped into both threads' memory. Both can read and write it simultaneously — which means you now *do* have real concurrency and must coordinate with `Atomics` (atomic loads/stores, `Atomics.wait` / `notify`) to avoid races.

Cost of creation matters. Spinning up a worker is not free — a new thread plus a new V8 isolate costs single-digit-to-tens of milliseconds and a few MB of memory. Creating one worker per request under load will bury you. The production pattern is a **worker pool**: a fixed set of long-lived workers that pull jobs from a queue, so you pay start-up once and reuse. Node 12.11+ ships `node:worker_threads` primitives, and libraries like `piscina` provide a battle-tested pool; you can also hand-roll one.

Note that some of Node's own async APIs already offload to libuv's thread pool without any worker_threads involvement — `crypto.pbkdf2`, `zlib`, `fs`, `crypto.randomBytes`. If a heavy operation has an async built-in variant, prefer it; worker_threads is for *your* JavaScript computation that has no such offload.

Code examples

Offload a CPU-bound calculation so the main thread's event loop stays responsive.

JAVASCRIPT

Copy

```
// main.js
import { Worker } from 'node:worker_threads';

function runHeavyJob(input) {
  return new Promise((resolve, reject) => {
    const worker = new Worker('./heavy.js', { workerData: input });
    worker.once('message', resolve);
    worker.once('error', reject);
    worker.once('exit', (code) => {
      if (code !== 0) reject(new Error(`Worker stopped with code ${code}`)
    });
  });
}

// The event loop keeps serving requests while this runs on another thread
const result = await runHeavyJob({ n: 45 });
console.log('fib:', result);
```

JAVASCRIPT

Copy

```
// heavy.js — runs on its own thread with its own heap
import { workerData, parentPort } from 'node:worker_threads';

function fib(n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }

const answer = fib(workerData.n);   // blocks THIS thread only
parentPort.postMessage(answer);     // send the result back, then exit
```

A minimal reusable worker **pool** — the pattern you actually ship. Fixed workers, a task queue, one worker per job at a time.

JAVASCRIPT

Copy

```
import { availableParallelism } from 'node:os';

export class Pool {
  #idle = [];
  #queue = [];
  constructor(file, size = availableParallelism()) {
    for (let i = 0; i < size; i++) this.#idle.push(new Worker(file));
  }
  run(data) {
    return new Promise((resolve, reject) => {
      this.#queue.push({ data, resolve, reject });
      this.#pump();
    });
  }
  #pump() {
    if (!this.#queue.length || !this.#idle.length) return;
    const worker = this.#idle.pop();
    const { data, resolve, reject } = this.#queue.shift();
    const done = (fn) => (v) => { worker.off('error', onErr); this.#idle.p
    const onErr = done(reject);
    worker.once('message', done(resolve));
    worker.once('error', onErr);
    worker.postMessage(data);
  }
}
```

Real shared memory with `SharedArrayBuffer` and `Atomics` — multiple threads incrementing one counter safely.

JAVASCRIPT Copy

```
// Shared counter across threads (no copy, no message per update)
import { Worker, isMainThread, workerData } from 'node:worker_threads';

if (isMainThread) {
  const shared = new SharedArrayBuffer(4);       // 4 bytes = one Int32
  const view = new Int32Array(shared);
  for (let i = 0; i < 4; i++) new Worker(new URL(import.meta.url), { worke
  setTimeout(() => console.log('total:', Atomics.load(view, 0)), 500);
```

```
} else {
  const view = new Int32Array(workerData);
  for (let i = 0; i < 100000; i++) Atomics.add(view, 0, 1);  // atomic +=
}
```

Common mistakes & gotchas

- **Creating a worker per request.** Thread + isolate startup is expensive; under load this melts down. Use a fixed pool and reuse workers.
- **Expecting shared variables by default.** Threads have separate heaps. A module-level variable in the worker is not the same object as in main. Only `SharedArrayBuffer` is truly shared; everything else is copied or transferred.
- **Using a transferred buffer after transfer.** After `postMessage(buf, [buf.buffer])`, the sender's buffer is detached — reading it throws or yields empty. Transfer is a hand-off, not a share.
- **Reaching for worker_threads for I/O.** I/O is already non-blocking on the main thread. Threading it adds overhead for no gain. Worker_threads is for *CPU*-bound JS. Use cluster (or just async I/O) for concurrency.
- **Forgetting to terminate workers.** Leaked workers keep threads and memory alive and can prevent the process from exiting. Call `worker.terminate()` when done (pools should reuse, but bound the pool and clean up on shutdown).
- **Ignoring the `'error'` and non-zero `'exit'` events.** An uncaught throw inside a worker surfaces as an `'error'` event, not a rejected promise, unless you wire it up. Unhandled, it can crash the worker silently.
- **Races on SharedArrayBuffer without Atomics.** Plain reads/writes to shared memory across threads are not safe. Use `Atomics` for correctness and for the wait/notify handshake.

Best practices

- **Pool, don't spawn per task.** Use `piscina` or a small fixed pool sized to `availableParallelism()`; queue work onto idle workers.

- **Move only what you must.** Send inputs, return outputs — don't ship huge objects back and forth. Transfer ArrayBuffers for large binary data; share via SharedArrayBuffer only when threads genuinely need live concurrent access.
- **Prefer built-in async offloads first.** `zlib`, `crypto.pbkdf2`, async `fs` already use libuv's pool. Reach for worker_threads only for your own CPU-bound JavaScript.
- **Keep workers pure and stateless** where possible — input in, result out — so they're easy to pool and restart.
- **Always handle** `error` **/** `exit` and set timeouts so a stuck job doesn't leak a worker.
- **Measure before threading.** If the op has an async built-in or the payload is tiny, the overhead of a worker can exceed the benefit.

⭐ Interview lens

Q: worker_threads vs cluster — what's the real difference and when do you use each?

A: cluster forks separate *processes* sharing a port to scale I/O concurrency across cores; they share no memory. worker_threads spawns *threads* inside one process to run CPU-bound JavaScript off the main thread, and they can share memory via SharedArrayBuffer. Rule of thumb: many concurrent requests → cluster; heavy computation per request → worker_threads.

Q: Does async/await help with CPU-bound work?

A: No. async/await only helps with *waiting* (I/O). A synchronous computation still runs to completion on the current thread and blocks the event loop regardless of async syntax. You need worker_threads (or a queue/external service) to get it off the main thread. (Very common trap.)

Q: How do you share data with a worker without copying it?

A: Two ways. Transfer an ArrayBuffer with the transfer list — zero-copy ownership hand-off, sender's buffer detaches. Or use a SharedArrayBuffer for genuinely

Q: Why is creating a worker per request a bad idea?
A: Each worker needs a new OS thread and a new V8 isolate — milliseconds of startup and megabytes of memory. Per-request creation collapses under load. Use a fixed-size worker pool that reuses long-lived workers pulling from a job queue.

Q: Do worker threads share the same event loop as the main thread?
A: No. Each worker has its own event loop, its own microtask/macrotask queues, and its own heap. Blocking one thread does not block another's loop — that isolation is the whole point.

Q: You need to hash passwords under heavy signup load. worker_threads or something else?
A: Prefer the built-in async offload first — `crypto.pbkdf2` / `scrypt` already run on libuv's thread pool, off the main thread, no worker_threads needed. Reach for worker_threads only for CPU-bound JS that has no async built-in. (Good senior answer names the async built-in.)

Cheat-card

- **worker_threads = threads in one process** for CPU-bound JS; each has its own loop + heap.
- Default message = **structured-clone copy**; transfer an `ArrayBuffer` = zero-copy; `SharedArrayBuffer` = real shared memory + `Atomics`.
- **cluster** = processes for I/O concurrency; **worker_threads** = threads for computation. Complementary.
- async/await does **nothing** for CPU-bound code — only worker_threads/queue gets it off the main thread.

- Never spawn per request → use a **fixed pool** (piscina) sized to `availableParallelism()`.
- Prefer built-in async offloads (`zlib`, `crypto.pbkdf2`, async `fs`) before threading.
- Handle `error` / `exit`; terminate/clean up workers on shutdown.

QUICK CHECK — no marks, just for you.

1. What is the core difference between cluster and worker_threads?

- ◯ cluster runs threads for CPU work; worker_threads run processes for I/O
- ◯ cluster forks separate processes to scale I/O concurrency across cores; worker_threads spawn threads inside one process to run CPU-bound JavaScript off the main thread
- ◯ They are identical; worker_threads is just the newer name for cluster
- ◯ cluster shares memory via SharedArrayBuffer; worker_threads never can

Check answer

2. Does using async/await help with CPU-bound work in Node?

- ◯ Yes, async/await automatically moves computation onto another thread
- ◯ Yes, awaiting a synchronous function yields to the event loop mid-calculation
- ◯ No, only cluster can help with any kind of work
- ◯ No — async/await only helps with waiting (I/O); a synchronous computation still runs to completion on the current thread and blocks the event loop

Check answer

3. Why is creating a worker thread per request a bad idea under load?

- Each worker needs a new OS thread and V8 isolate (ms of startup, MBs of memory), so per-request creation collapses under load — use a fixed pool instead
- Worker threads cannot be created more than once per process
- The main thread blocks while any worker exists
- Node forbids more than one worker at a time

Check answer

Previous
The cluster module

Next
Child processes

LEARN MORE

ARTICLES & DOCS

- Node.js Worker Threads (worker_threads)
  nodejs.org
- Node.js Cluster module (process-level vs thread-level)
  nodejs.org

VIDEOS



Multithreading in Node.js with Worker Threads Explained
logicBase Labs

Hand-picked references for this lesson.